



**INSTITUTO DE EDUCACIÓN SECUNDARIA LA MOLA
NOVELDA**

DESARROLLO DE APLICACIONES WEB

Curso Académico 2025/2026

Desarrollo Web en Entorno Cliente

**U01_A03_CPE Caso práctico extendido – Prof. Emilio
Ramírez**

Autor: Segura Abad, Mario

Fecha: 26 de Septiembre, 2025

ÍNDICE

INTRODUCCIÓN3

ENUNCIADO.....4

 Trabajo con proyectos de NodeJS4

DESARROLLO DE LA PRACTICA.....5

 Descripción5

 Contenidos del archivo5

 • Código del Apartado 1 de teoría:5

 • Scripts adjuntos:6

 • Interacción con funciones matemáticas:7

 • Validación de resultados:.....9

 • Documentación técnica:.....10

 Cómo visualizarlo.....10

 1. Descargar o clonar el proyecto desde el repositorio o archivo comprimido.....10

 2. Abrir los archivos .js en un editor compatible, como Visual Studio Code, Sublime Text o Atom.....10

 3. Ejecutar los programas desde la consola utilizando NodeJS:10

 4. Revisar las capturas de ejecución incluidas en la carpeta correspondiente, que muestran los resultados por consola, la pantalla del VS Code con el propio código y la autoría (fecha y nombre del autor).10

 5. Consultar el documento PDF para entender, asimismo, el desarrollo completo del caso práctico y la estructura del proyecto.10

 Objetivo de la práctica.....11

 • Comprender la estructura básica de un proyecto NodeJS.11

 • Implementar funciones matemáticas simples en módulos reutilizables.11

 • Validar el comportamiento del código mediante pruebas unitarias con el módulo assert11

 • Ejecutar scripts desde consola y analizar los resultados obtenidos.....11

 • Documentar correctamente cada ejercicio, incluyendo nombre, fecha y capturas de pantalla. 11

 • Familiarizarse con la creación de documentación técnica en formato Markdown y PDF.11

 • Aplicar buenas prácticas de programación, como la separación de responsabilidades y el uso de comentarios identificativos y explicativos.11

 • “Reflexionar” sobre la importancia de las pruebas en el desarrollo de software y cómo contribuyen a mejorar la calidad del código.11

CONCLUSIÓN12

Autor.....12

INTRODUCCIÓN

En este ejercicio denominado “Trabajo con proyectos NodeJS”, gracias a los archivos proporcionados en la propia tarea, deberemos completar el ejercicio o caso práctico extendido ejemplificando los contenidos.

ENUNCIADO

Trabajo con proyectos de NodeJS

Caso práctico que ejemplifica los contenidos.

Descargad los archivos de proyecto para completar el caso práctico extendido.

DESARROLLO DE LA PRACTICA

Práctica DWEK – Unidad 2

Descripción

Este proyecto contiene varios ejercicios prácticos relacionados con el **Apartado 1 de la teoría de NodeJS**, centrados en el desarrollo de funciones matemáticas básicas y su validación mediante pruebas unitarias.

El objetivo principal es aplicar los conceptos fundamentales de NodeJS vistos en clase, como la modularización del código, el uso de módulos nativos como `assert`, y la ejecución de scripts desde consola.

Además, se ha trabajado la documentación técnica del proyecto, tanto en formato Markdown como PDF, siguiendo las instrucciones de entrega establecidas. Se han incluido comentarios identificativos en cada archivo fuente, capturas de pantalla de la ejecución, y una estructura clara y profesional en todo los apartados.

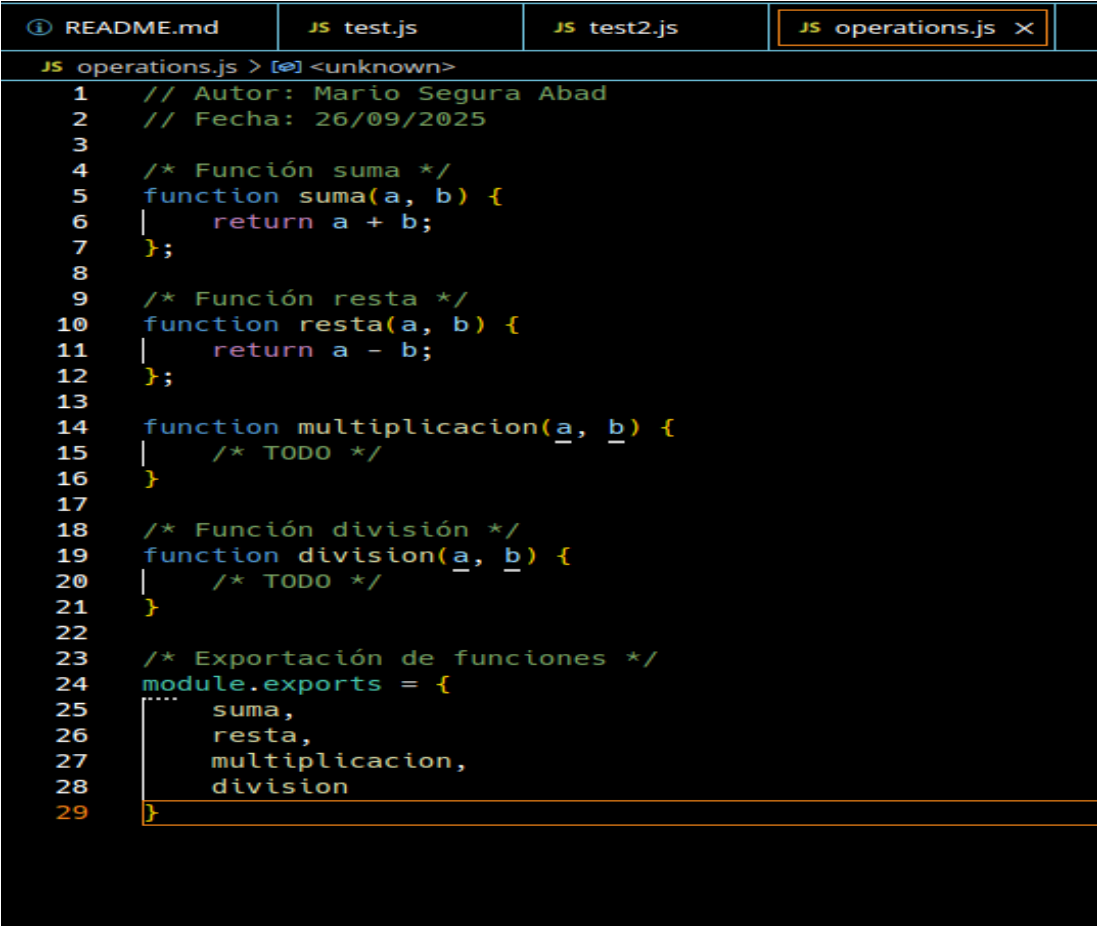
Contenidos del archivo

En el proyecto se pueden encontrar los siguientes apartados:

- **Código del Apartado 1 de teoría:**

Se ha implementado un módulo externo `operation.js`, que contiene funciones matemáticas básicas como `suma` y `division`. Estas funciones están diseñadas para ser reutilizables e ideales para realizar tests. Cada archivo incluye comentarios con el nombre y el autor y la fecha de realización.

Operations.js original en VS Code:

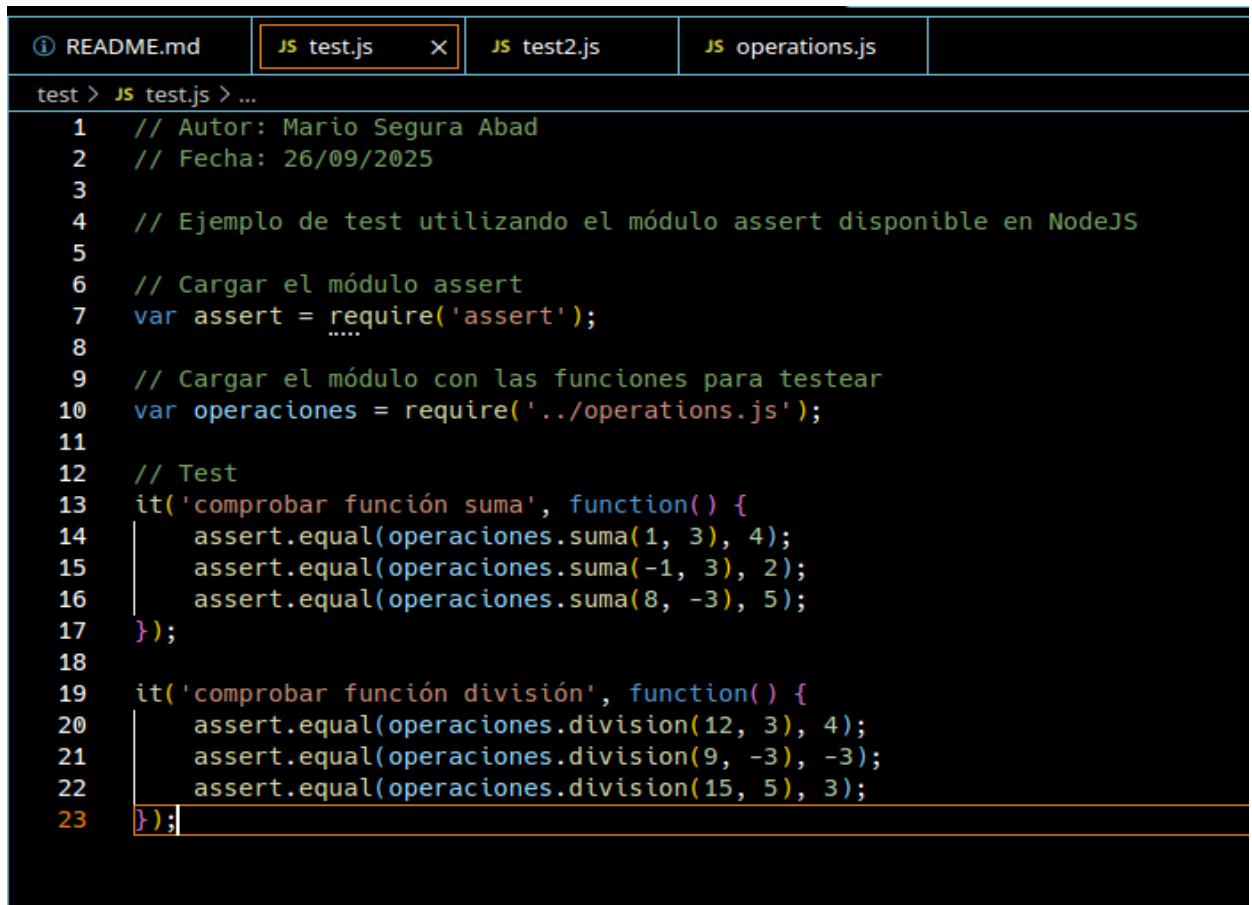


```
JS operations.js > [?] <unknown>
1  // Autor: Mario Segura Abad
2  // Fecha: 26/09/2025
3
4  /* Función suma */
5  function suma(a, b) {
6  |    return a + b;
7  };
8
9  /* Función resta */
10 function resta(a, b) {
11 |    return a - b;
12 };
13
14 function multiplicacion(a, b) {
15 |    /* TODO */
16 }
17
18 /* Función división */
19 function division(a, b) {
20 |    /* TODO */
21 }
22
23 /* Exportación de funciones */
24 module.exports = {
25 |    suma,
26 |    resta,
27 |    multiplicacion,
28 |    division
29 }
```

- **Scripts adjuntos:**

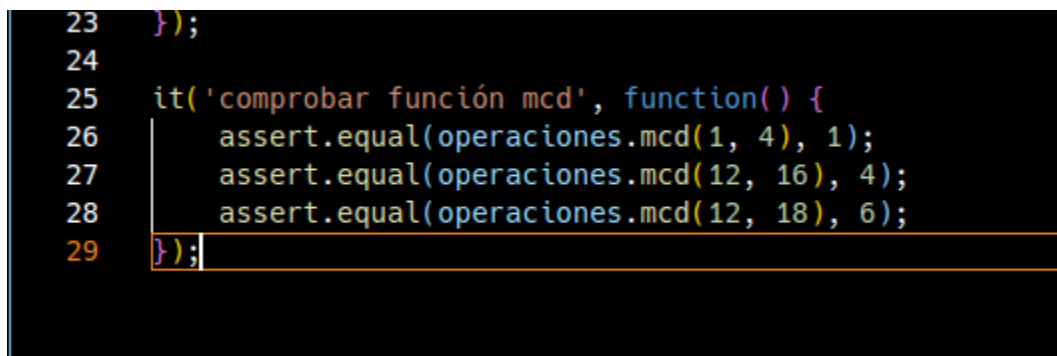
Se ha creado un archivo de pruebas que utiliza el módulo assert de NodeJS para comprobar el correcto funcionamiento de las funciones. Se han definido varios casos de prueba que cubren diferentes combinaciones de valores, incluyendo dos funciones nuevas (mcd y mcm) para asegurar la fiabilidad del código.

Test.js original en VS Code:



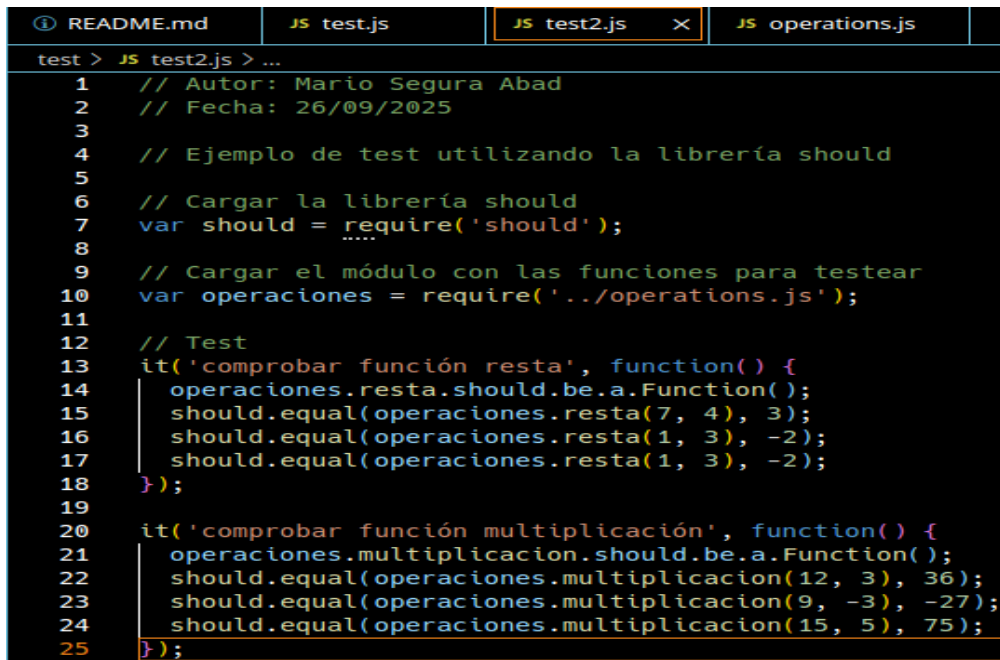
```
test > JS test.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 26/09/2025
3
4  // Ejemplo de test utilizando el módulo assert disponible en NodeJS
5
6  // Cargar el módulo assert
7  var assert = require('assert');
8
9  // Cargar el módulo con las funciones para testear
10 var operaciones = require('../operations.js');
11
12 // Test
13 it('comprobar función suma', function() {
14     assert.equal(operaciones.suma(1, 3), 4);
15     assert.equal(operaciones.suma(-1, 3), 2);
16     assert.equal(operaciones.suma(8, -3), 5);
17 });
18
19 it('comprobar función división', function() {
20     assert.equal(operaciones.division(12, 3), 4);
21     assert.equal(operaciones.division(9, -3), -3);
22     assert.equal(operaciones.division(15, 5), 3);
23 });
```

Test.js con una operación nueva en VS Code:



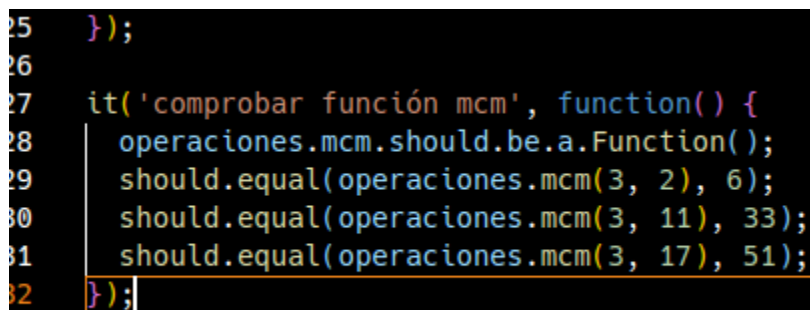
```
23 });
24
25 it('comprobar función mcd', function() {
26     assert.equal(operaciones.mcd(1, 4), 1);
27     assert.equal(operaciones.mcd(12, 16), 4);
28     assert.equal(operaciones.mcd(12, 18), 6);
29 });
```

Test2.js original en VS Code:



```
test > JS test2.js > ...
1  // Autor: Mario Segura Abad
2  // Fecha: 26/09/2025
3
4  // Ejemplo de test utilizando la librería should
5
6  // Cargar la librería should
7  var should = require('should');
8
9  // Cargar el módulo con las funciones para testear
10 var operaciones = require('../operations.js');
11
12 // Test
13 it('comprobar función resta', function() {
14   operaciones.resta.should.be.a.Function();
15   should.equal(operaciones.resta(7, 4), 3);
16   should.equal(operaciones.resta(1, 3), -2);
17   should.equal(operaciones.resta(1, 3), -2);
18 });
19
20 it('comprobar función multiplicación', function() {
21   operaciones.multiplicacion.should.be.a.Function();
22   should.equal(operaciones.multiplicacion(12, 3), 36);
23   should.equal(operaciones.multiplicacion(9, -3), -27);
24   should.equal(operaciones.multiplicacion(15, 5), 75);
25 });
```

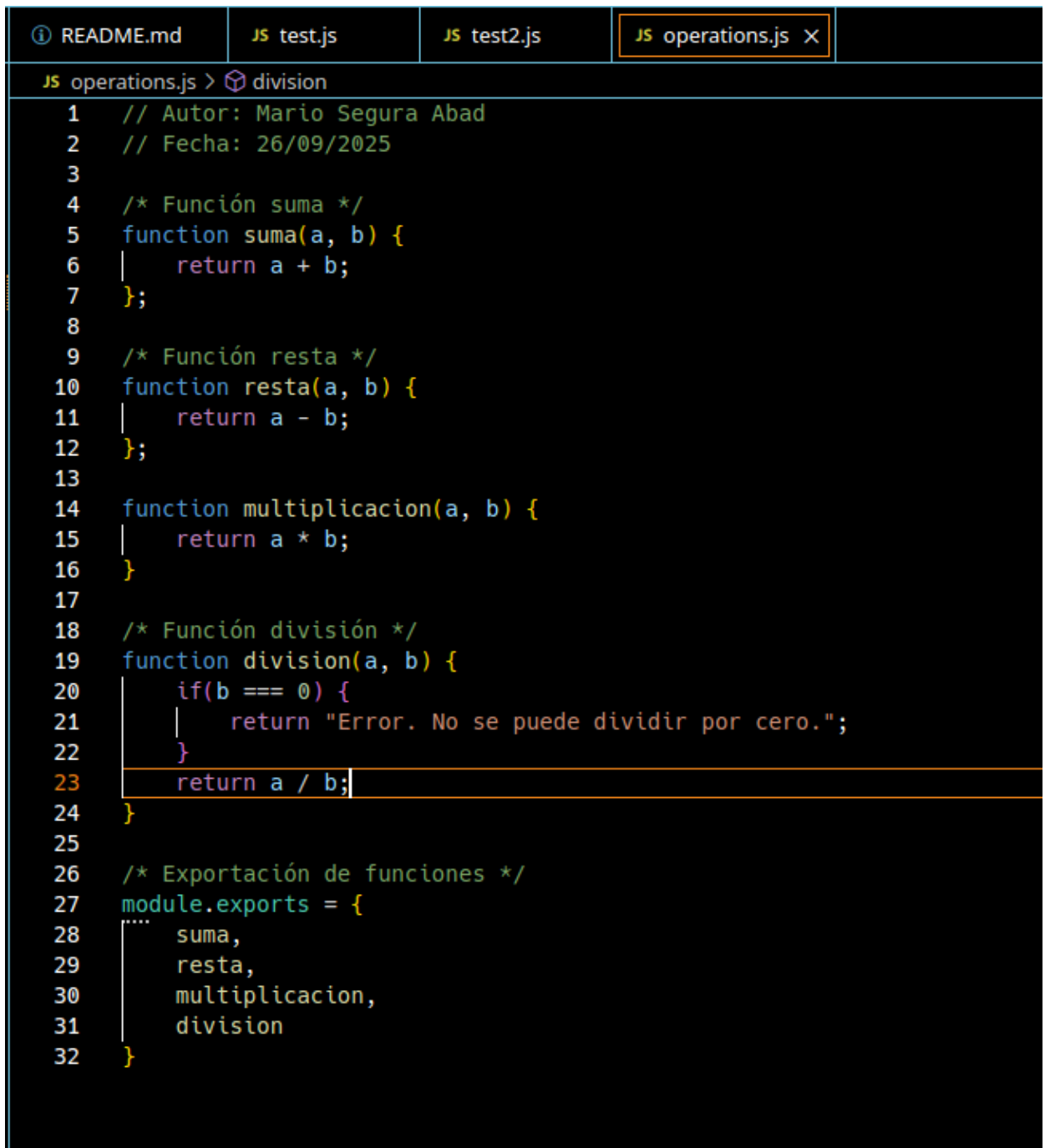
Test2.js con una operación nueva en VS Code:



```
25  });
26
27  it('comprobar función mcm', function() {
28   operaciones.mcm.should.be.a.Function();
29   should.equal(operaciones.mcm(3, 2), 6);
30   should.equal(operaciones.mcm(3, 11), 33);
31   should.equal(operaciones.mcm(3, 17), 51);
32  });
```

- **Interacción con funciones matemáticas:**

El módulo `operations.js` ha sido utilizado en un script de test que valida los resultados esperados. Este ejercicio demuestra cómo separar la lógica de negocio en módulos independientes y cómo integrarlos en scripts de prueba. Se han añadido capturas de pantalla que muestran la ejecución correcta de los tests en consola.

Operations.js con multiplicación y división completas en VS Code:

The image shows a screenshot of the Visual Studio Code editor interface. At the top, there is a tab bar with four tabs: 'README.md', 'JS test.js', 'JS test2.js', and 'JS operations.js'. The 'JS operations.js' tab is selected and highlighted with a yellow border. Below the tab bar, the editor displays the content of 'JS operations.js'. The code is written in JavaScript and includes comments in Spanish. It defines four functions: 'suma' (addition), 'resta' (subtraction), 'multiplicacion' (multiplication), and 'division' (division). The 'division' function includes a check for division by zero. At the bottom, there is an object 'module.exports' that exports the four functions. The code is as follows:

```
1 // Autor: Mario Segura Abad
2 // Fecha: 26/09/2025
3
4 /* Función suma */
5 function suma(a, b) {
6 |   return a + b;
7 };
8
9 /* Función resta */
10 function resta(a, b) {
11 |   return a - b;
12 };
13
14 function multiplicacion(a, b) {
15 |   return a * b;
16 }
17
18 /* Función división */
19 function division(a, b) {
20 |   if(b === 0) {
21 |     return "Error. No se puede dividir por cero.";
22 |   }
23 |   return a / b;
24 }
25
26 /* Exportación de funciones */
27 module.exports = {
28 |   suma,
29 |   resta,
30 |   multiplicacion,
31 |   division
32 }
```


Operations.js con un par de funciones nuevas en VS Code:

```
}

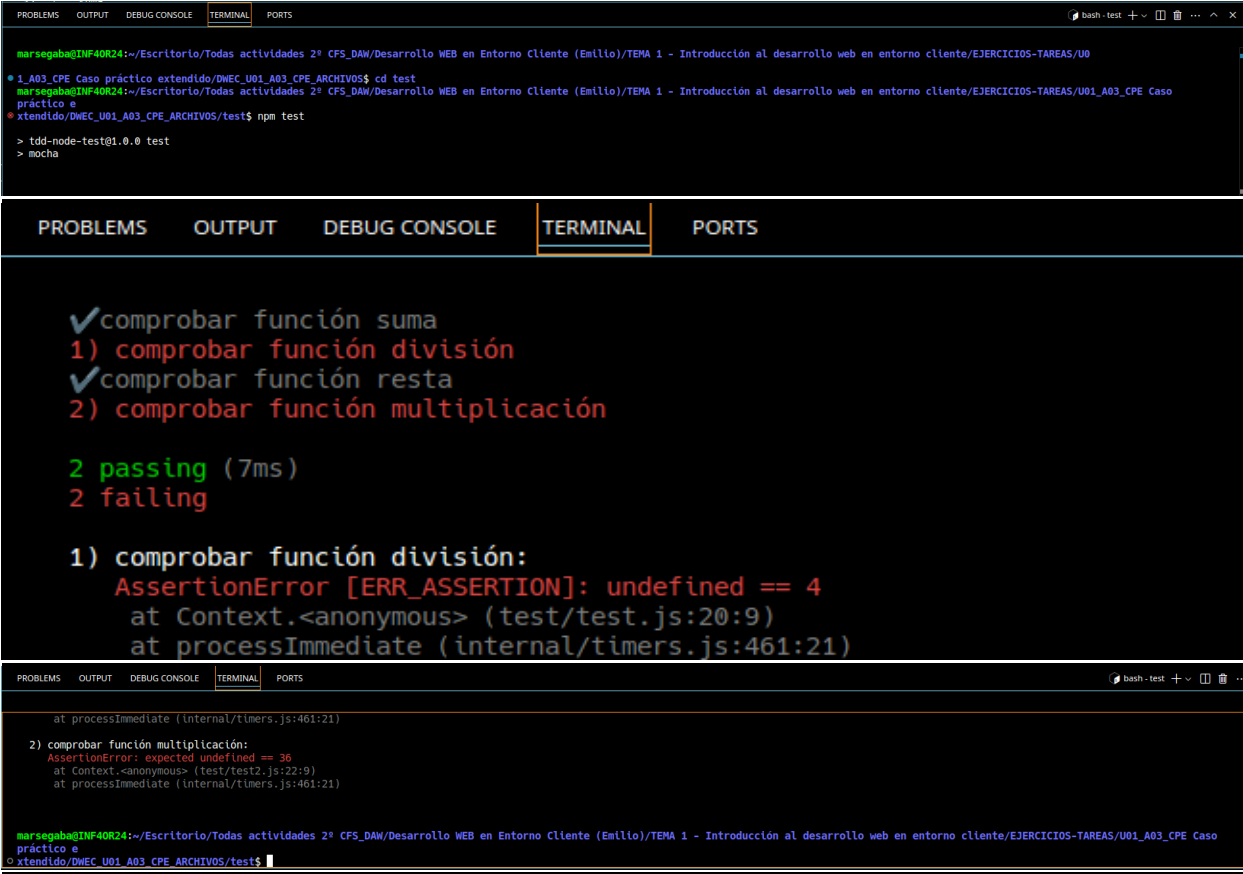
/* Función mcd */
function mcd(a, b) {
  while(b != 0) {
    aux = b;
    b = a % b;
    a = aux;
  }
  return a;
}

/* Función mcm */
function mcm(a, b) {
  return (a * b) / mcd(a, b);
}

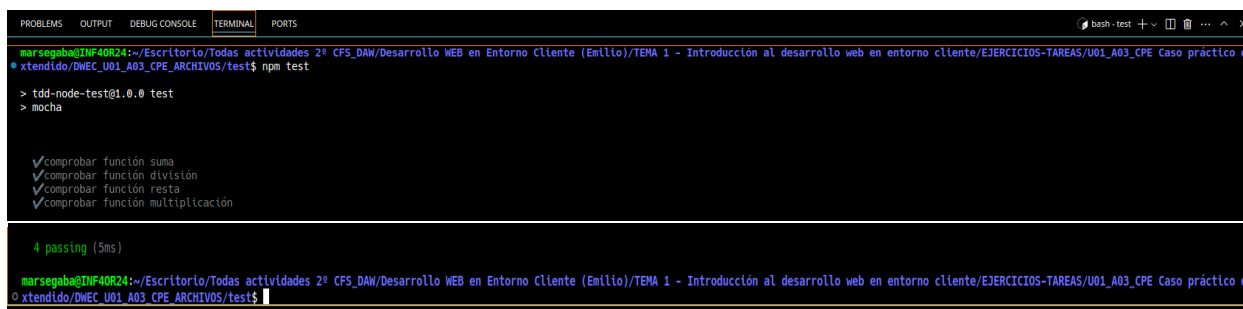
/* Exportación de funciones */
module.exports = {
  suma,
  resta,
  multiplicacion,
  division,
  mcd,
  mcm
}
```

- **Validación de resultados:**
Se ha realizado un análisis detallado de los resultados obtenidos en consola, identificando posibles errores y proponiendo mejoras. Este apartado permite “reflexionar” sobre la importancia de las pruebas unitarias y cómo ayudan a detectar fallos antes de desplegar el propio código.

Salida por consola sin modificaciones fallando:



Salida por consola sin modificaciones pasando:



```
marsegaba@INF40R24:~/Escritorio/Todas actividades 2º CFS_DAW/Desarrollo WEB en Entorno Cliente (Emilio)/TEMA 1 - Introducción al desarrollo web en entorno cliente/EJERCICIOS-TAREAS/U01_A03_CPE Caso práctico e
xtendido/DWEC_U01_A03_CPE_ARCHIVOS/test$ npm test

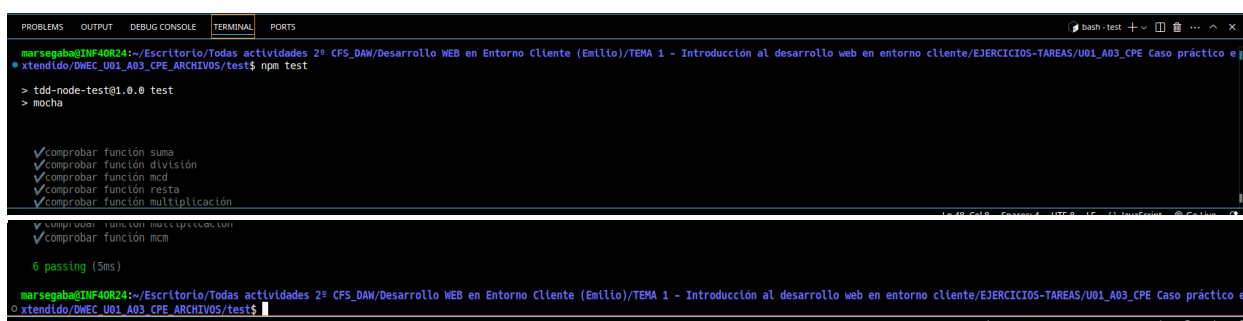
> tdd-node-test@1.0.0 test
> mocha

✓comprobar función suma
✓comprobar función división
✓comprobar función resta
✓comprobar función multiplicación

4 passing (5ms)

marsegaba@INF40R24:~/Escritorio/Todas actividades 2º CFS_DAW/Desarrollo WEB en Entorno Cliente (Emilio)/TEMA 1 - Introducción al desarrollo web en entorno cliente/EJERCICIOS-TAREAS/U01_A03_CPE Caso práctico e
xtendido/DWEC_U01_A03_CPE_ARCHIVOS/test$
```

Salida por consola con modificaciones pasando:



```
marsegaba@INF40R24:~/Escritorio/Todas actividades 2º CFS_DAW/Desarrollo WEB en Entorno Cliente (Emilio)/TEMA 1 - Introducción al desarrollo web en entorno cliente/EJERCICIOS-TAREAS/U01_A03_CPE Caso práctico e
xtendido/DWEC_U01_A03_CPE_ARCHIVOS/test$ npm test

> tdd-node-test@1.0.0 test
> mocha

✓comprobar función suma
✓comprobar función división
✓comprobar función mcd
✓comprobar función resta
✓comprobar función multiplicación
✓comprobar función mcm

6 passing (5ms)

marsegaba@INF40R24:~/Escritorio/Todas actividades 2º CFS_DAW/Desarrollo WEB en Entorno Cliente (Emilio)/TEMA 1 - Introducción al desarrollo web en entorno cliente/EJERCICIOS-TAREAS/U01_A03_CPE Caso práctico e
xtendido/DWEC_U01_A03_CPE_ARCHIVOS/test$
```

- **Documentación técnica:**

Se ha generado un documento PDF que incluye portada, introducción, desarrollo por ejercicios, conclusión y capturas de pantalla. Todo el contenido estará justificado y estructurado, cumpliendo con los requisitos del Anexo 1 – Instrucciones de entrega.

Cómo visualizarlo

Para visualizar y ejecutar correctamente el proyecto, se deben seguir los siguientes pasos:

1. **Descargar o clonar el proyecto** desde el repositorio o archivo comprimido.
2. **Abrir los archivos .js** en un editor compatible, como Visual Studio Code, Sublime Text o Atom.
3. **Ejecutar los programas** desde la consola utilizando NodeJS:
 - Navegar hasta la carpeta del proyecto.
 - Ejecutar el comando `node test/test.js` para lanzar los tests.
4. **Revisar las capturas de ejecución** incluidas en la carpeta correspondiente, que muestran los resultados por consola, la pantalla del VS Code con el propio código y la autoría (fecha y nombre del autor).
5. **Consultar el documento PDF** para entender, asimismo, el desarrollo completo del caso práctico y la estructura del proyecto.

Objetivo de la práctica

Los objetivos principales de esta práctica son:

- Comprender la estructura básica de un proyecto NodeJS.
- Implementar funciones matemáticas simples en módulos reutilizables.
- Validar el comportamiento del código mediante pruebas unitarias con el módulo `assert`.
- Ejecutar scripts desde consola y analizar los resultados obtenidos.
- Documentar correctamente cada ejercicio, incluyendo nombre, fecha y capturas de pantalla.
- Familiarizarse con la creación de documentación técnica en formato Markdown y PDF.
- Aplicar buenas prácticas de programación, como la separación de responsabilidades y el uso de comentarios identificativos y explicativos.
- “Reflexionar” sobre la importancia de las pruebas en el desarrollo de software y cómo contribuyen a mejorar la calidad del código.

CONCLUSIÓN

Esta práctica ha permitido aplicar los fundamentos básicos de NodeJS mediante la creación de funciones matemáticas muy simples y su propia validación con pruebas unitarias usando la palabra reservada `assert` .

Se ha comprobado el correcto funcionamiento del código en diversos casos, reforzando así la importancia de realizar los tests antes de desplegar cualquier modificación en el código.

Además, se ha documentado el proyecto siguiendo los requisitos técnicos y de presentación adecuado, lo que ha contribuido a una entrega clara, completa y profesional, (o eso espero).

Autor

Mario Segura Abad

Fecha de realización: 26/09/2025